Vibe Coding 脚手架化 · 方法论总览

把 AI 当工程师用，先给它规则、记忆和方法论

八条核心心法

AI 没记忆	→ 文档是它的记忆	AI 默认动手	→ 逼它先想
AI 默认讨好	→ 逼它给判断	AI 默认结论	→ 逼它给证据
AI 默认前进	→ 逼它会停	判断模糊先聊	规则清晰才做
改代码前先改文档	→ 改完代码再核文档	AI 默认不清理	→ 逼它善后

善后 = 墓碑代码用完即清：临时测试 / 调试打印 / 临时测试接口 / 写死假数据 / 一次性脚本
删前先列清单经用户确认；正式回归测试和有意保留的运行日志不删

四道门禁 · 项目推进的硬关卡

每道门禁必须用证据证明规则立住了，全过才进入下一阶段

立项门 方向定死了 / AI 有规矩了 / 立项 + 选型 + 宪法 → **架构门** 四条线跑通了 / 真源文档有了 / 前端+数据库+后端+验收 → **业务门** 功能做完了 / 安全过了 / 业务开发 + 安全审查 → **上线门** 验收过 / 反哺了

每道门禁的过门标准（硬指标）

立项门	立项文档 8 项内容 · 技术选型唯一推荐 · Agent 宪法落盘 · Git 提交 ≥3 次
架构门	前端能启动 · 数据库能建表查询 · 后端 4 条线跑通 · 真源文档落盘 · Git ≥4 次
业务门	P0 功能完成 · 每轮有验收证据 · 安全 5 关卡全过 · 水平越权测试 · 无拼接注入
上线门	5 维度审查无严重项 · 风险项登记 · 验收报告落盘 · 项目上线 · 至少反哺 1 条 Skill
	墓碑代码已清：临时测试/调试打印/临时接口/假数据/一次性脚本，先列清单确认后删
	勿删：正式回归测试 + 有意保留用于排查问题的运行日志

核心原则
没证据的项目标记"未验证"，不允许"基本完成"。要么"通过"，要么"未通过"加原因。

14 阶段全流程时间线

从纯想法到上线反哺，每阶段一个可验收的交付块

阶段 1 · 项目立项

触发：准备开始新项目

产出：立项文档（8 项内容） · 验收：AI 确认"暂时不写代码" + Git 提交

阶段 2 · 技术选型

触发：立项文档已生成

产出：技术选型文档 · 验收：AI 给唯一推荐 + 能说清"为什么不选其他"

阶段 3 · 架构设计文档

触发：选型确定

产出：目录结构 / 接口规则 / 错误处理 / 日志规则 · 验收：大纲确认后落盘

阶段 4 · Agent 宪法

触发：架构设计完成

产出：AGENT_CONSTITUTION.md · 验收：≤200 行 + 禁止事项 + 专属条款 ≥5 + 代码卫生

阶段 5 · 实施真源文档拆解

触发：宪法落盘

产出：首个大阶段拆解 · 验收：子阶段 + 步骤 + 每步有验收标准

阶段 6 · 前端骨架

触发：立项、选型、宪法都已完成

产出：前端实施文档 + 最小可运行骨架 · 验收：能启动 + 设计 token + UI 库接入

阶段 7 · 数据库设计

触发：前端骨架完成

产出：schema.sql + 实施计划 · 验收：能建表/插入/查询 + 三大范式校验

阶段 8 · 后端骨架

触发：数据库设计完成

产出：后端架构实施文档 + 骨架 · 验收：启动/接口/业务/运维四条线跑通

阶段 9 · 后端联调

触发：骨架跑通

产出：联调验证报告 · 验收：前端调通 + 跨域 + 错误场景验证

阶段 10 · 架构验收

触发：联调通过

产出：验收报告 + 实施真源文档 · 验收：11 种接口场景 + 启动证据包完整

阶段 11 · 业务模块开发（多轮）

触发：架构验收通过

产出：业务接口 + 每轮 Git 提交 · 验收：一轮一验收（含墓碑代码善后清理）

阶段 12 · 安全审查

触发：业务功能开发后

产出：安全边界表 + 权限设计表 + 整改清单 · 验收：5 关卡全过 + 无拼接注入

阶段 13 · 上线前总验收

触发：所有功能开发完成

产出：上线前验收报告 · 验收：5 维度审查 + 墓碑代码体检（先清单后删）+ 无严重项

阶段 14 · 上线 + Skill 反哺

触发：验收通过，准备上线

产出：复盘报告 + 迭代后 Skill 库 · 验收：至少反哺 1 条新经验

紫色=立项门 · 蓝色=架构门 · 琥珀=业务门 · 青绿=上线门，颜色对应上图四道门禁

12 个 Skill 方法论矩阵

语义命中后加载的"专业技能"，按 需求设计 / 验收审查 / 调试测试 三类组织

需求与设计

brainstorming
头脑风暴
"我想做个 XX"时触发

project-breakdown
功能拆解
立项后拆功能清单

writing-plans
编写开发计划
拆任务/锁范围时触发

database-design
数据库设计
建表前先出设计文件：业务对象 / 关系图 / 核心表 / 三大范式 / schema.sql

验收与审查

verification-before-completion
完成前验证
AI 说"已完成"前必走 + 善后清理

requesting-code-review
代码审查
上线前 5 维度审查 + 墓碑残留检查

browser-verification
浏览器验证
前端验证/跨端时触发

backend-architecture-acceptance
后端架构验收
骨架搭好后验收时触发

backend-security-review
后端安全审查
安全审查 5 关卡时触发

frontend-security-review
前端安全审查
XSS/CSRF/敏感信息

调试与测试

systematic-debugging
系统化调试
找根因不猜 · 连续 3 轮修不好 → 刹车 → 换新对话带真源 → 逼根因后再动手

testing-strategy
测试六步法
手动闭环→三类用例→回归网→假测试陷阱→攻防→防过度；临时测试用完即删，回归网长期保留

Skill 的三个设计目的

1. 保持宪法简洁	不要什么都往 Agent 宪法里塞，具体流程放 Skill
2. 减少 AI 漂移	让 AI 在 Vibe Coding 过程中不每次都自由发挥
3. 技术平权	让小白也能借用高手的工程经验，按需调用而非记忆

3. 技术干货	让小白也能借用高手的工程经验，按需调用而非记忆

Skill 复制到项目根目录 skills/，在 Agent 宪法里引用，AI 才能读到

项目状态决策树与知识库结构

拿到项目先判断状态，再走对应流程；知识库按"知识库+工作台分离"组织

第一步：判断项目处于什么状态

决策路径：有代码吗？→ 能跑起来吗？→ 有真源文档吗？→ 代码和文档对得上吗？

状态 0	状态 1	状态 2	状态 3	状态 4
纯想法	烂尾	半成品	黑盒	健康
没代码没文档	有代码跑不起来	能跑但功能残缺	能跑但没文档	能跑+文档对得上
→ 走 0-1 完整流程	→ 烂尾抢救流程	→ 半成品补全流程	→ 黑盒考古流程	→ 健康项目迭代

关键提示：90% 的"已完成项目"实际是状态 3（黑盒），不是状态 4。先按黑盒考古流程走。

上线前新增两道体检：① 墓碑代码清理（临时测试/调试打印/临时接口/假数据/脚本，先清单后删，勿删回归测试与运行日志）
② 反复修不好止损（连续 3 轮失败 → 换新对话带真源 → 逼根因后再动手）

知识库结构 · 知识库与工作台分离

知识库放规则方法论（查证用），工作台放门禁卡状态追踪（每天干活用）

templates/ · 工作台模板
4 张门禁卡 + PROJECT_STATUS
只复制 PROJECT_STATUS.md；门禁卡留在知识库查看
复制会冻结旧版，改版不生效

skills/ · 方法论库
12 个 Skill 方法论
项目启动时复制到根目录
在 Agent 宪法里引用

docs/ · Prompt模板库
12 部分 + 50+ 个模板
含墓碑代码清理 / 止损话术
分挖掘型/执行型/验收型三类

docs/ · 深度手册
4 篇深度操作手册
遇到盲区时翻对应章节查证
可视化流程/操作手册/Prompt库

workflows/ · 流程速查
14 阶段合并为 1 个速查
回顾流程时查某阶段快速步骤
含墓碑代码体检步骤

sources/ · 原始笔记
21 篇原始笔记归档
需要溯源某条规则来源时翻
偶尔翻

日常使用心法

日常维护项目里的	PROJECT_STATUS.md；知识库只翻 templates/ 和 skills/
新项目第一天	复制 skills/ + PROJECT_STATUS 到项目根目录，翻「立项门」照着做
遇到盲区	翻 docs/AI协作全流程操作手册 对应章节 + docs/全流程Prompt模板库

版本管理：知识库本身用 Git，只增不删，加规则必加场景，反哺前先验证